Steps 1 to 6, Explained

Intent-Based SDN for 5G Service-Level Agreement Enforcement
Team 16 | Development Record

6 of 14 subtasks complete | 248 passing host tests
ST-1 through ST-6 verified on a live virtual network in Docker

Written for a reader starting from zero. Every step answers what we are doing, why, how, when and where. Every tool is justified. Nothing is claimed that has not been observed.

PART 0

The problem, in plain English

A mobile network carries very different traffic over the same wires. A remote-surgery video feed must never stutter. A file download just needs to be fast on average. A field of sensors sends a few bytes an hour. They share the same cables and switches.

The operator sells promises about quality. A promise looks like: *"this video call will never be delayed more than 20 milliseconds, will always get at least 20 megabits per second, and will lose no more than half a percent of packets."* That is a **Service-Level Agreement**, or SLA.

The gap: network equipment does not understand promises. A switch understands only *"packets that look like this go out of port 3."* Somebody must translate.

Three things go wrong today

Translation is manual	An engineer converts the promise into switch configuration by hand. Slow, and two engineers do it differently.
Enforcement is open-loop	Once installed, nothing checks the promise is still kept. Traffic shifts, a link degrades, the promise quietly breaks. Nobody notices until a customer complains.
Requests are accepted blindly	A system that cannot compute remaining capacity will accept a customer it cannot serve, damaging customers it already had. This is silent failure, and it is the worst of the three.

What the system does today, and what it will do

IMPLEMENTED AND OBSERVED LIVE

Working now, steps 1 to 6. You write the promise in a small structured file a computer can read. The system validates it, computes a route that satisfies the constraints, installs bidirectional OpenFlow 1.3 rules and a bandwidth-limiting queue on the switches, and then measures the delay, jitter and loss the traffic actually receives. When the measurements breach the promise, it raises an SLA violation.

STILL TO COME

Not yet built, step 7. Nothing currently connects a detected violation to an automatic reroute. The detector raises the alarm; no component acts on it. Automatic closed-loop recovery is the goal of this project, not a capability it has today.

Why this is worth building

Comparable research systems are generally reported with substantial infrastructure requirements, which makes independent reproduction difficult. This project targets a different point: a reference implementation that runs on a single Docker-capable laptop, so the results can be re-run and checked.

We claim no new invention. We claim a working reference implementation with an honest evaluation. That is a legitimate contribution and one we can defend.

PART 1

Every tool we chose, and why

A reasonable question in any review is "why that one?" Here is the answer for each. The versions listed are the ones the built container reports.

Mininet 2.3.1b4

What it is	Software that creates a whole network of switches and computers inside one laptop. Not a maths simulation - it runs real Linux processes and pushes real packets down virtual links.
Why we chose it	Physical switches cost thousands and we have none. Mininet gives genuine packet-forwarding behaviour on a laptop, which is the entire point of the project.
What we rejected	A pure simulator such as ns-3. Rejected because it models packets mathematically rather than forwarding them, so behaviour would not transfer to real equipment.

Open vSwitch 2.17.12

What it is	A network switch implemented in software. Mininet uses it for every switch in our topology. This is a virtual switch, not physical hardware.
Why we chose it	The same switch software used in production data centres. It speaks OpenFlow properly and supports the HTB queues we use to enforce a bandwidth guarantee.
What we rejected	Mininet's simpler built-in switch. Rejected - it does not support the queueing we need.

OpenFlow 1.3

What it is	The protocol a controller uses to install forwarding rules on switches.
Why we chose it	It is the widely supported standard and carries the match fields we need. Our implementation uses OpenFlow 1.3 flow rules together with OVS HTB queues; we do not use OpenFlow meters or multi-table pipelines.
What we rejected	Versions 1.4 and 1.5. Rejected - switch support is patchy and we gain nothing.

Ryu 4.34

What it is	The central program that decides what every switch should do. Written in Python.
Why we chose it	Our whole system is Python, so one language throughout. Ryu has a small API and many tutorials, which matters for a team learning SDN for the first time.
What we rejected	ONOS, a production Java controller. More future-proof, but adds a Java toolchain and a steep learning curve to a schedule with no slack. Recorded as our documented fallback.

Python 3.9.25 (pinned)

What it is	An older Python version, fixed deliberately inside the container.
Why we chose it	Ryu depends on eventlet, which breaks on Python 3.10 and newer. This is a known, documented failure. Pinning 3.9 inside the container avoids it entirely.
What we rejected	Using the system Python. Rejected - it is 3.10 on modern Ubuntu and Ryu will not run.

Docker

What it is	A container: a sealed environment holding an operating system and exact versions of every tool.
Why we chose it	It makes the environment reproducible - the same versions on any Docker-capable machine. It also isolates the fragile Python 3.9 pin from the host.
What we rejected	A virtual machine. Heavier and slower; a container is sufficient.

NetworkX

What it is	A Python library for graphs - nodes joined by edges. Our network is a graph: switches are nodes, links are edges.
Why we chose it	Route finding is a graph problem. NetworkX provides tested shortest-path machinery so we are not debugging our own.
What we rejected	Writing our own graph code. Rejected - solved, standard, and not where our effort belongs.

FastAPI

What it is	A library for building the northbound web interface, so an operator submits a promise over HTTP instead of editing files on the machine.
Why we chose it	Fast to write, validates inputs, and generates its own documentation page - useful in a live demonstration.
What we rejected	Flask. Similar, but FastAPI's automatic validation saves work.

SQLite

What it is	A database held in a single local file. No server to install or run.
Why we chose it	It gives local single-file persistence for intents, paths and measurements. Raw experiment evidence such as baseline CSV files is written separately under experiments/results/raw so it can be inspected without opening the database.
What we rejected	PostgreSQL or MySQL. Rejected - they need a running server, which works against a one-command start.

pytest and ruff

What it is	The tools that run our automated tests and check code style.
Why we chose it	With several people building interlocking parts, tests are how a breaking change is found in seconds rather than in November. The suite now stands at 248 tests, and ruff reports no findings.
What we rejected	Manual testing. Rejected - it does not scale and does not catch regressions.

PART 2

The six steps

The project is 14 one-week subtasks. Steps 1 to 6 are implemented and tested. Each below answers the five questions, then states plainly what exists and what does not.

STEP 1 07-13 Aug | DONE - 50 tests

Set up the workshop and freeze the vocabulary

WHAT	We built the container with exact tool versions, and fixed the exact vocabulary an operator may use to write a promise.
WHY	Two reasons. If versions drift the project stops building halfway through and a week disappears. If the vocabulary keeps growing, every new word means new work in all six modules - our own risk list calls this the single biggest threat to finishing.
HOW	A Dockerfile pinning the toolchain. The vocabulary is a JSON Schema: a machine-readable rulebook stating exactly which fields are legal.
WHEN	Week 1. It must be first - everything else is built inside this container and speaks this vocabulary.
WHERE	Dockerfile, requirements.txt, src/intent_manager/schema.json.

WHAT NOW EXISTS

The Docker image builds successfully. The running container reports Python 3.9.25, Mininet 2.3.1b4, Open vSwitch 2.17.12 and Ryu 4.34.

The vocabulary is frozen at **five things you may promise** (minimum bandwidth, maximum delay, maximum loss, isolation from another tenant, links to avoid), **three responses to a broken promise** (reroute, degrade, alert only), and **one priority scale** from 1 to 5.

A validator that accepts a promise or rejects it with a precise pointer to the mistake. It does not crash on malformed input - tested with numbers, empty files, broken text and wrong shapes.

Eight example promises, one per scenario, and six deliberately broken ones.

50 tests, including tests that fail if anyone widens the vocabulary. The freeze is enforced by the test suite, not by a note in a document.

SCOPE AND LIMITS

Enforcement uses OpenFlow 1.3 flow rules and OVS HTB queues. OpenFlow meters and multi-table pipelines are not used.

STEP 2 14-20 Aug | DONE - 19 tests, started live

Build the test network

WHAT	We defined the network the system manages: 7 switches, 8 hosts, and the capacity and delay of every link.
WHY	We need somewhere to work. Critically we need **three link-disjoint routes** between the two important hosts. The system's job is to move traffic somewhere better when a promise breaks - with one route there is nowhere to move, and the behaviour cannot be tested.
HOW	One Python file holds every switch, host and link as data. The Mininet script and the tests both read that file, so what the tests check is by construction what gets built.
WHEN	Week 2, immediately after the toolchain is fixed.
WHERE	topology/topology_spec.py, topology/team16_topo.py.

WHAT NOW EXISTS

The seven-switch Mininet topology was started successfully in the Docker environment and its routes exercised.

Three routes from s1 to s7, deliberately unequal so a change of route is visible: **northern** s1-s2-s3-s7 at 100 Mbps and 6 ms, **middle** s1-s4-s5-s7 at 50 Mbps and 15 ms, **southern** s1-s6-s7 at 40 Mbps and 24 ms.

Only the northern route satisfies the baseline promise; the southern breaks its latency bound outright. That asymmetry is what makes a route change observable.

A test that **proves** the three routes share no link, rather than assuming it.

A finding: the original plan's diagram was ambiguous about two links, and without them only one route exists and the plan's own claim was false. We resolved it, marked both inferred, and a test proves the resolution.

STEP 3 21-27 Aug | DONE - 32 tests

Agree the shared language between the six parts

WHAT	We fixed the exact shapes of data the six modules exchange, before writing any of them.
WHY	Several people build interlocking parts at once. If one person's idea of a path differs from another's, nothing fits, and the mismatch surfaces late. Our risk register names this and names this step as the mitigation.
HOW	One package, src/common, holding the shared shapes: an intent, a path, a link, a measurement, an admission decision. Plus the database layout, the event format, and the audit log.
WHEN	Week 3. Deliberately before any module is written. The step that looks like a delay and is not.
WHERE	src/common/models.py, events.py, db.py, audit.py.

WHAT NOW EXISTS

The anti-oscillation rule lives in **exactly one place**, so two modules cannot disagree about whether it is safe to move traffic again.

A decision object that **refuses to exist without a reason**. Constructing a rejection with an empty explanation raises an error, which enforces "no silent failures" at the type level.

SQLite gives local single-file persistence for intents, paths, measurements and events. Raw experiment evidence, such as the baseline CSV, is written separately under experiments/results/raw.

The database records **every** path ever installed rather than overwriting, so a route change is two rows. That makes "how many times did it move?" countable directly.

An audit log recording the full chain: what was measured, what was promised, what was decided, what was done.

STEP 4 28 Aug - 03 Sep | DONE - 32 tests

Accept a promise and compute a route

WHAT	The system accepts a written promise over a REST interface and computes a route through the switches that satisfies its constraints.
WHY	The first step where the system produces something a person can inspect. Until now it was foundations.
HOW	Three pieces. Yen's k-shortest-paths finds candidate routes. A pruning stage discards any route that breaks the promise, **recording a plain-English reason for each rejection**. A selection stage picks the best survivor, deterministically so runs repeat.
WHEN	Week 4, once the shared language existed.
WHERE	src/pathing/, src/intent_manager/api.py.

WHAT NOW EXISTS

Submit a promise over the REST API, receive a computed route, in roughly half a millisecond.

Refusals explain themselves: `exceeds max_latency_ms: 24 > 20` and `insufficient bandwidth: 100 < 500`.

Endpoints to submit, list, inspect and withdraw a promise, plus a health check.

Demonstrable with `python demo_st4.py` and through the REST API.

Two real defects caught by these tests: the route computation raised an exception on an unknown address instead of explaining itself, and the database connection would have failed once the web server dispatched a request to its worker thread.

SCOPE AND LIMITS

ST-4 computes and stores a route. It does not by itself program any switch. Enforcement begins at ST-5.

STEP 5 04-10 Sep | DONE - 42 tests, run live

Program the switches

WHAT	Code that converts a computed route into OpenFlow 1.3 rules and an OVS HTB queue, and installs them on the switches through the Ryu controller.
WHY	A route in a database forwards no packets. This is where the system stops being a calculator and becomes a network controller.
HOW	Three parts: a translator turning a route into per-switch rules; a queue manager creating an HTB queue that enforces the bandwidth guarantee; and the Ryu application that installs them.
WHEN	Week 5, after routes could be computed.
WHERE	src/enforcement/flowmod.py, queues.py, controller.py.

WHAT NOW EXISTS

Bidirectional OpenFlow 1.3 rules were generated and installed by Ryu onto live virtual Open vSwitch switches.

Each intent carries a **cookie**, a numeric tag stamped on its rules so they can be located and removed later without disturbing anyone else's.

IPv4 matches always set `eth_type 0x0800`. A rule that matches on IP protocol without it installs cleanly and then never matches anything - the most common silent failure in this area.

Output ports come from the port map the controller discovers at startup; they are never guessed.

A **20 Mbps HTB queue was attached to s1-eth2** and confirmed present.

Traffic was observed traversing the northern route s1-s2-s3-s7.

42 tests covering cookie determinism, rule counts and direction, the eth_type requirement, port-map discipline and queue command units.

SCOPE AND LIMITS

This was verified on a **virtual software datapath** - Open vSwitch running under Mininet inside Docker. **Physical networking hardware has not been tested.**

STEP 6 11-17 Sep | DONE - 48 tests, run live

Measure what the traffic actually receives

WHAT	Continuously measure the delay, jitter, loss and throughput each promise is actually receiving, and decide when a promise has genuinely been broken.
WHY	Without measurement the system is open-loop: it configures once and never checks. Measurement is what makes this a control system rather than a configuration script.
HOW	Two sources of numbers, then four filters. Switch counters read periodically give loss and throughput; small timestamped UDP probes give delay and jitter. Then: an exponentially weighted average so one noisy reading is ignored; a dead-band so a value hovering at the limit does not flicker; a rule requiring several consecutive bad readings; and a hold-off timer so the system does not react twice in quick succession.
WHEN	Week 6. It needs step 5's installed rules to have something to measure.
WHERE	src/telemetry/detector.py, collector.py, probes.py, runtime.py.

WHAT NOW EXISTS

The collector read counters from the live virtual datapath, and UDP probes supplied delay, jitter and loss.

In the live Scenario S1 run, delay measured between roughly **6.3 and 7.1 ms** against a 20 ms promise, with **0 percent probe loss**. **Five measurements were persisted**, four of them complete samples.

The detector moved through green, amber and red states and emitted an `sla.violated` event.

The deciding logic is **completely separate** from the measuring, so it can be exercised with scripted numbers and no network. That design choice is why this step carries 48 tests.

48 tests, including proof that a single bad reading does not trigger, that a value resting on the limit emits nothing, and that recovery requires clearing the far side of the dead-band.

A result that needs explaining honestly

In the live run the violation that fired was on **minimum bandwidth**, and it fired because of how the scenario is built, not because the network underperformed.

What happened	Scenario S1 sends small telemetry probes, not a sustained 20 Mbps workload. Counter-derived throughput was therefore close to zero, and after the required consecutive samples the minimum-bandwidth promise was correctly detected as violated.
What this proves	That measurement, persistence and violation detection work end to end on a live datapath. The pipeline observed a real condition and reacted to it.
What this does NOT prove	That the system delivers 20 Mbps of application throughput under load. No sustained-bandwidth test has been run. What was verified is that the 20 Mbps HTB queue is correctly configured on s1-eth2.
What is needed next	A sustained traffic generator driving the path at its guaranteed rate, so throughput is measured under realistic load rather than inferred from probe traffic.

Baseline measurement, for comparison

Before any intent logic runs, a control condition is measured on the same topology - plain forwarding, no promises, no monitoring. Everything later is compared against it.

Packet loss	0 percent
Round-trip time	minimum 12.213 ms, average 12.825 ms, maximum 26.945 ms, mdev 1.460 ms
Throughput	71.7 Mbits/sec
Evidence	experiments/results/raw/b0_baseline_20260906134918.csv

PART 3

Honest status

Stated plainly, because being caught overclaiming costs more than admitting a gap.

Subtasks completed	ST-1 through ST-6. **6 of 14**, roughly 43 percent by count.
Host verification	**248 passed**, 2 warnings. Ruff reports no findings.
Container verification	**237 passed, 11 skipped**, 3 warnings. The skips are tests that need hardware or host-only conditions.
Live result	Docker plus Mininet plus Open vSwitch plus Ryu. **Scenario S1 passed**: intent accepted, rules installed with the expected cookie, HTB queue attached, northern route traversed, measurements persisted.
Physical hardware	**Not tested.** All live verification used a virtual software datapath.
ST-7, automatic rerouting	**Next, and not complete.** Some unit-tested groundwork exists, but no verified automatic reroute follows a detected violation.
Dashboard	**Not implemented.** It is ST-11.
Sustained bandwidth	**Not tested.** Only probe-level traffic has been measured.
Novelty	None claimed. This is a reference implementation and an evaluation.

Test breakdown

Intent validation	50
Telemetry	48
Enforcement	42
Shared interfaces	32
Routing and REST API	32
Control loop	25
Topology	12
Topology script	7
Total	**248**

The one thing that is still missing

Step 6 can detect that a promise is broken. Step 5 can install a route. **Nothing verified yet joins them.** The detector raises an alarm and no component is confirmed to act on it.

Step 7 is that single connection, and it is the project's central claim in one step. Groundwork exists and is unit tested, but until a detected violation produces an automatic reroute on the live datapath, ST-7 is not complete.

What the next subtasks add

ST-7	**Close the loop.** Connect a detected violation to an automatic reroute that avoids the links which just failed.

ST-8	Admission control. Before accepting a new promise, check there is capacity. If not, refuse with a stated reason or displace something less important - never accept and fail silently.
ST-9	Preemption and tuning of the damping filters, so the oscillation comparison can be produced with damping on and off.
ST-11	The dashboard and audit view. Not started.

Summary for an examiner. Six of fourteen subtasks are complete. An operator can submit a machine-readable SLA intent, the system validates it, computes a constraint-satisfying route, installs bidirectional OpenFlow 1.3 rules and an OVS HTB queue, and measures the resulting live virtual datapath. Scenario S1 passed in Docker using Mininet, Open vSwitch and Ryu. The next subtask is ST-7, which will connect detected SLA violations to automatic rerouting. Physical hardware has not been tested.

PART 4

Words you will need

Learn these well enough to define in one sentence if interrupted.

SDN	Software-Defined Networking. Normally every switch decides for itself where to send packets. In SDN that decision is taken away and given to one central program.
Switch	A device that receives a packet on one port and forwards it out another. Ours are virtual switches running as software.
Link	A connection between two switches. Our links carry a capacity in megabits per second and a delay in milliseconds.
Port	A numbered socket on a switch. A forwarding rule says which port a packet leaves by.
Datapath	The part of a switch that actually moves packets, as opposed to the control software that programs it. Ours is a virtual software datapath, not physical hardware.
Controller	The central program that programs the switches. Ours is Ryu.
OpenFlow	The protocol the controller uses to install rules on switches. We use version 1.3.
Flow rule	One instruction inside a switch: packets matching this pattern leave by that port. A switch holds many.
Cookie	A numeric tag stamped on every rule belonging to one intent, so those rules can be found and removed later without touching anyone else's.
HTB queue	Hierarchical Token Bucket. A queue on a switch port that limits or guarantees a traffic rate. This is how a minimum-bandwidth promise is enforced.
REST API	A way for one program to ask another to do something over HTTP. Ours accepts an intent and returns the computed route.
Northbound API	The interface an operator or higher-level system uses to talk to the controller - the way intents come in. Southbound is the opposite direction, controller to switches, which is OpenFlow.
Intent	The promise, written in a structured file a computer can read.
SLA	Service-Level Agreement. The business promise about quality. Our intents are machine-readable SLAs.
Closed loop	Measure, notice something is wrong, fix it, measure again - with no human involved. Open loop means configure once and never check. We are open loop until ST-7.
Telemetry	The measurements collected while running: delay, jitter, loss, throughput.
Jitter	How much the delay varies. A steady 20 ms is fine for a video call; alternating between 5 and 40 ms is not.
EWMA	Exponentially weighted moving average. A running average weighting recent readings more heavily, so one odd sample does not move it much.
Hysteresis	A deliberate gap between the level that raises an alarm and the level that clears it, so a value on the boundary does not flicker.
Dwell timer	A hold-off. Having just moved traffic, do not move it again for N seconds however bad it looks.
Counter delta	The difference between two readings of a switch counter. Rates must be computed from differences, never from the absolute value.
Link-disjoint	Two routes sharing no link. If one link fails, the other route is unaffected.

Baseline	A deliberately simpler configuration measured for comparison, so an improvement can be shown rather than asserted.
URLLC / eMBB / mMTC	Three 5G service categories: ultra-reliable low-latency, enhanced mobile broadband, and massive machine-type communication. These are categories of service, not applications.
telemedicine-video	A sample tenant name used in our example intents. It represents a latency-sensitive, reliability-sensitive use case. It is not a network protocol, and it is not the same thing as URLLC - URLLC is the 5G service category such a use case would fall under.
Admission control	Checking there is capacity before accepting a new promise, refusing with a reason rather than accepting and failing silently. This is ST-8 and is future work.